

Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

ОТЧЕТ

по расчетно-графической работе
по дисциплине «Базы данных»
«Проектирование и реализация базы данных»

Тема:

«Система учета сетевой топологии и передачи трафика в локальной сети»

Выполнили:

студенты группы ИКС-433

Криволапов Н.

Добромилов А.

Проверил:

Брагин К.И.

Новосибирск 2026

Содержание

1. Цель работы
2. Описание предметной области
3. Логическая модель базы данных
4. Реализация структуры базы данных
5. Ограничения целостности
6. Индексы
7. Представления
8. Функции и триггеры
9. Тестовые данные
10. Функциональные SQL-запросы
11. Контейнеризация
12. Проверка работоспособности
13. Соответствие требованиям задания
14. Заключение

1. Цель работы

Целью расчетно-графической работы является проектирование и реализация реляционной базы данных в СУБД PostgreSQL для предметной области, связанной с телекоммуникационными технологиями и IT-инфраструктурой.

В рамках работы требуется описать предметную область, спроектировать структуру базы данных, реализовать таблицы, ключи и ограничения, добавить индексы, представления, функции и триггеры, подготовить тестовое наполнение и набор аналитических запросов.

2. Описание предметной области

В качестве предметной области выбрана система учета сетевой топологии и передачи трафика в локальной сети. Такая система может использоваться как часть NMS (Network Management System) для хранения информации об узлах сети, коммутаторах, портах, подключениях, MAC-таблице и событиях передачи Ethernet-кадров.

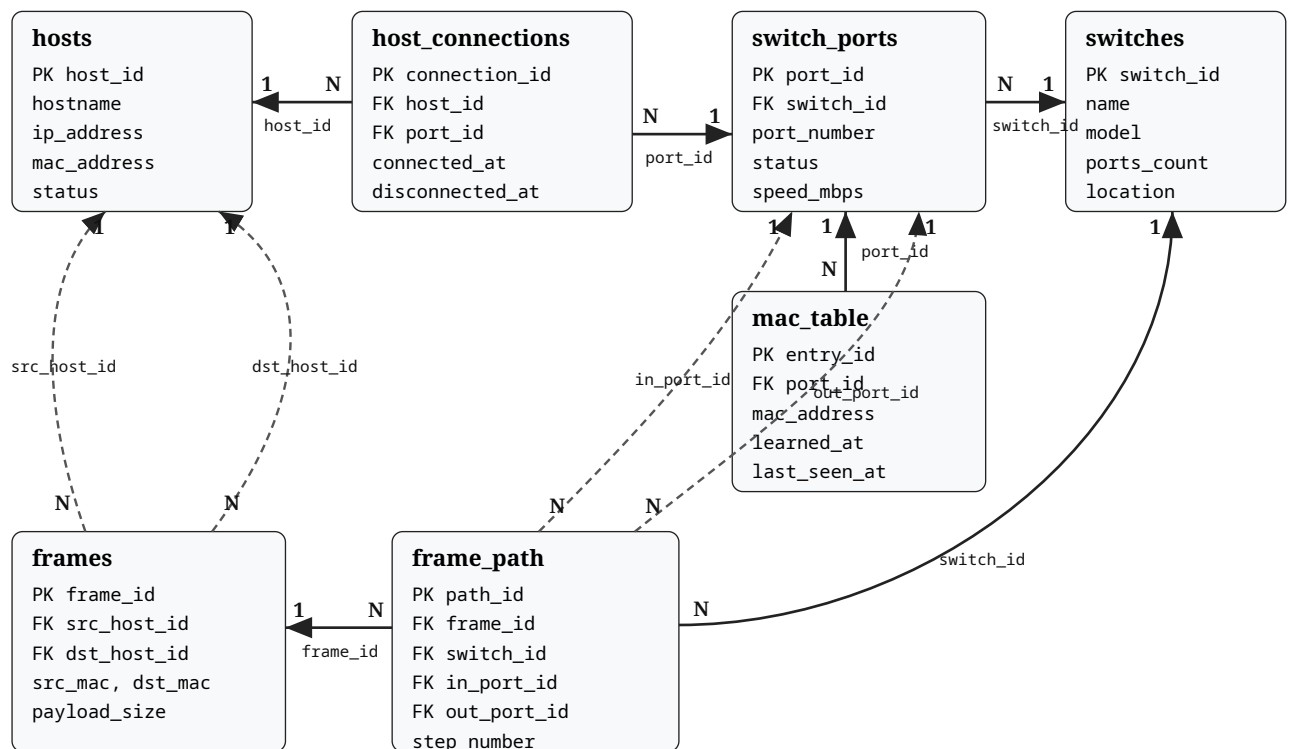
В локальной сети рабочие станции и серверы подключаются к портам коммутаторов. Коммутатор выполняет пересылку Ethernet-кадров между портами, используя MAC-таблицу. MAC-таблица содержит сведения о том, какой MAC-адрес был замечен на каком порту. При передаче кадра система фиксирует отправителя, получателя, MAC-адреса, тип кадра, размер полезной нагрузки и путь прохождения кадра через сетевое оборудование.

Основные автоматизируемые задачи:

- ведение реестра сетевых хостов;
- учет коммутаторов и их портов;
- учет активных подключений хостов к портам;
- хранение MAC-таблицы коммутаторов;
- регистрация фактов передачи кадров;
- хранение пути прохождения кадров через порты коммутаторов;
- построение аналитических отчетов по топологии и трафику.

3. Логическая модель базы данных

База данных содержит семь основных таблиц: `hosts`, `switches`, `switch_ports`, `host_connections`, `mac_table`, `frames`, `frame_path`. Таблицы нормализованы: сведения о хостах, коммутаторах, портах, подключениях и событиях передачи хранятся отдельно и связываются внешними ключами.



Обозначения: стрелка идет от внешнего ключа к связанной таблице; 1 — одна родительская запись, N — много дочерних записей.

Рисунок 1 — ER-диаграмма базы данных с внешними ключами и кратностями связей

Связи между таблицами:

Дочерняя таблица и внешний ключ	Родительская таблица и первичный ключ	Кратность	Смысл связи
host_connections.host_id	hosts.host_id	N:1	Один хост может иметь несколько записей истории подключений.
host_connections.port_id	switch_ports.port_id	N:1	Один порт может встречаться в истории подключений.
switch_ports.switch_id	switches.switch_id	N:1	Один коммутатор содержит много портов.
mac_table.port_id	switch_ports.port_id	N:1	На одном порту может быть изучено несколько MAC-адресов.
frames.src_host_id	hosts.host_id	N:1	Хост может быть отправителем многих кадров.
frames.dst_host_id	hosts.host_id	N:1	Хост может быть получателем многих кадров.
frame_path.frame_id	frames.frame_id	N:1	Один кадр может иметь несколько шагов пути.
frame_path.switch_id	switches.switch_id	N:1	Один коммутатор может участвовать во многих шагах прохождения кадров.
frame_path.in_port_id	switch_ports.port_id	N:1	Порт может быть входным для многих шагов прохождения кадров.
frame_path.out_port_id	switch_ports.port_id	N:1	Порт может быть выходным для многих шагов прохождения кадров.

4. Реализация структуры базы данных

Структура таблиц реализована в файле sql/001_init.sql. Для каждой сущности задан первичный ключ. Межтабличные связи реализованы с помощью внешних ключей.

Таблица	Назначение	Ключевые поля
hosts	Конечные устройства сети	host_id, hostname, ip_address, mac_address, status
switches	Сетевые коммутаторы	switch_id, name, model, ports_count
switch_ports	Порты коммутаторов	port_id, switch_id, port_number, status, speed_mbps
host_connections	Подключения хостов к портам	connection_id, host_id, port_id, connected_at
mac_table	MAC-таблица коммутаторов	entry_id, port_id, mac_address, last_seen_at
frames	Факты передачи Ethernet-кадров	frame_id, src_host_id, dst_host_id, src_mac, dst_mac
frame_path	Путь кадра через сетевое оборудование	path_id, frame_id, switch_id, in_port_id, out_port_id

```
CREATE TABLE hosts (  
    host_id SERIAL PRIMARY KEY,  
    hostname VARCHAR(100) NOT NULL,  
    ip_address INET UNIQUE NOT NULL,  
    mac_address MACADDR UNIQUE NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE switch_ports (  
    port_id SERIAL PRIMARY KEY,  
    switch_id INT NOT NULL REFERENCES switches(switch_id),  
    port_number INT NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    speed_mbps INT NOT NULL,  
    UNIQUE (switch_id, port_number)  
);
```

5. Ограничения целостности

Для обеспечения корректности данных используются ограничения NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY и дополнительные ЧЕК-ограничения из файла sql/003_db_objects.sql.

Ограничение	Назначение
chk_hosts_status	Разрешает только статусы online, offline, maintenance.
chk_switches_ports_count	Запрещает неположительное количество портов у коммутатора.
chk_switch_ports_status	Ограничивает статус порта значениями up, down, disabled.
chk_switch_ports_speed	Проверяет, что скорость порта больше нуля.
chk_frames_hosts_are_different	Запрещает отправку кадра от хоста самому себе.
chk_frames_payload_size	Проверяет положительный размер полезной нагрузки.
chk_frames_delivery_status	Ограничивает статус доставки значениями delivered, dropped, flooded.
chk_frame_path_ports_are_different	Запрещает одинаковые входной и выходной порты для одного шага пути.

6. Индексы

Индексы ускоряют частые операции поиска, фильтрации и соединения таблиц. В базе созданы индексы для поиска по статусам, внешним ключам, времени отправки кадров и трассировке пути.

Индекс	Обоснование
idx_hosts_status	Ускоряет выборку хостов по статусу.
idx_switch_ports_switch_id	Ускоряет получение портов конкретного коммутатора.
idx_host_connections_host_id	Ускоряет поиск подключений конкретного хоста.
idx_host_connections_port_id	Ускоряет поиск подключений по порту.
idx_mac_table_mac_address_unique	Обеспечивает уникальность MAC-адреса и ускоряет поиск.
idx_frames_src_host_id	Ускоряет отчеты по отправителям кадров.
idx_frames_dst_host_id	Ускоряет отчеты по получателям кадров.
idx_frames_sent_at	Ускоряет аналитику по времени передачи.
idx_frame_path_frame_id	Ускоряет трассировку пути конкретного кадра.

```
CREATE INDEX IF NOT EXISTS idx_frames_src_host_id ON frames(src_host_id);
CREATE INDEX IF NOT EXISTS idx_frames_dst_host_id ON frames(dst_host_id);
CREATE INDEX IF NOT EXISTS idx_frames_sent_at ON frames(sent_at);
CREATE INDEX IF NOT EXISTS idx_frame_path_frame_id ON frame_path(frame_id);
```

7. Представления

Представление	Назначение
v_active_connections	Показывает активные подключения хостов к портам коммутаторов.
v_switch_mac_table	Показывает MAC-таблицу с названиями коммутаторов и номерами портов.
v_frame_trace	Показывает путь прохождения кадров через коммутаторы.
v_host_traffic_stats	Сводит статистику отправленных и полученных кадров по хостам.

8. Функции и триггеры

Для реализации серверной логики используются функции PostgreSQL на SQL и PL/pgSQL. Функции автоматизируют обновление MAC-таблицы, регистрацию кадров и получение аналитических отчетов.

Функция	Назначение
learn_mac_address(...)	Добавляет или обновляет запись в MAC-таблице.
register_frame(...)	Регистрирует кадр и автоматически создает запись о его пути.
get_frame_trace(...)	Возвращает трассировку конкретного кадра.
get_host_traffic_stats()	Возвращает статистику трафика по хостам.
get_switch_mac_table(...)	Возвращает MAC-таблицу указанного коммутатора.

```
SELECT register_frame(1, 2, 'IPv4', 128, 'delivered') AS new_frame_id;
```

Триггер	Назначение
trg_validate_frame_hosts	Проверяет, что MAC-адреса кадра соответствуют хостам.
trg_learn_src_mac_from_frame	После вставки кадра обновляет MAC-таблицу по адресу отправителя.
trg_validate_frame_path_ports	Проверяет, что входной и выходной порты пути принадлежат указанному коммутатору.

9. Тестовые данные

Тестовые данные находятся в файле sql/002_seed.sql. В базу добавлены три хоста, один коммутатор, три порта, активные подключения, начальная MAC-таблица, пример переданного кадра и путь кадра.

```
INSERT INTO hosts (hostname, ip_address, mac_address, status) VALUES
('Workstation-01', '192.168.1.10', '00:50:56:c0:00:01', 'online'),
('Workstation-02', '192.168.1.11', '00:50:56:c0:00:02', 'online'),
('Server-Main', '192.168.1.200', '00:50:56:c0:00:ff', 'online');
```

10. Функциональные SQL-запросы

Файл sql/queries.sql содержит 20 демонстрационных запросов. Они покрывают простые выборки, фильтрацию, соединения таблиц, агрегацию, HAVING, CTE, сортировку, ограничение результата, временную аналитику и оконные функции.

10.1 Активные подключения

```
SELECT *
FROM v_active_connections
ORDER BY switch_id, port_number;
```

10.2 MAC-таблица коммутатора

```
SELECT *
FROM get_switch_mac_table('Core-Switch-01');
```

10.3 Трассировка кадра

```
SELECT *
FROM get_frame_trace(1);
```

10.4 Статистика трафика

```
SELECT *
FROM get_host_traffic_stats();
```

10.5 Запрос с HAVING

```
SELECT
    h.hostname,
    COUNT(f.frame_id) AS frames_sent,
    SUM(f.payload_size) AS bytes_sent
FROM hosts h
JOIN frames f ON f.src_host_id = h.host_id
GROUP BY h.hostname
HAVING COUNT(f.frame_id) > 1
ORDER BY frames_sent DESC;
```

10.6 Запрос с CTE и LIMIT

```
WITH traffic_pairs AS (
    SELECT
        src.hostname AS src_hostname,
        dst.hostname AS dst_hostname,
        COUNT(*) AS frames_count,
        SUM(f.payload_size) AS total_bytes
    FROM frames f
    JOIN hosts src ON src.host_id = f.src_host_id
    JOIN hosts dst ON dst.host_id = f.dst_host_id
    GROUP BY src.hostname, dst.hostname
)
SELECT *
FROM traffic_pairs
ORDER BY frames_count DESC, total_bytes DESC
LIMIT 3;
```

10.7 Дополнительные запросы 13-20

Для расширения аналитического набора добавлены запросы, показывающие свободные ресурсы сети, распределение трафика, последние события и оконную нумерацию кадров.

Номер	Назначение запроса	Используемые конструкции
13	Хосты без активного подключения к коммутатору	LEFT JOIN, WHERE IS NULL
14	Свободные порты коммутаторов	JOIN, LEFT JOIN, фильтрация
15	Количество кадров по статусам доставки	GROUP BY, COUNT, SUM
16	Динамика трафика по дням	DATE, GROUP BY, AVG
17	Последние 5 переданных кадров с именами хостов	JOIN, ORDER BY, LIMIT
18	Утилизация портов коммутаторов	GROUP BY, агрегаты, вычисляемые поля
19	MAC-адреса на портах без активного подключения	LEFT JOIN, WHERE IS NULL
20	Нумерация кадров по каждому отправителю	ROW_NUMBER(), PARTITION BY

11. Контейнеризация

Для запуска PostgreSQL используется Docker Compose. При первом старте контейнера автоматически выполняются SQL-файлы инициализации.

```

services:
  postgres:
    image: postgres:16
    container_name: lan_traffic_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: lan_traffic_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./sql/001_init.sql:/docker-entrypoint-initdb.d/001_init.sql:ro
      - ./sql/002_seed.sql:/docker-entrypoint-initdb.d/002_seed.sql:ro
      - ./sql/003_db_objects.sql:/docker-entrypoint-initdb.d/003_db_objects.sql:ro

```

Запуск проекта:

```
docker compose up -d
```

Подключение к базе:

```
docker exec -it lan_traffic_postgres psql -U postgres -d lan_traffic_db
```

Полная очистка контейнера и тома данных:

```
docker compose down -v
```

12. Проверка работоспособности

В ходе проверки были созданы 7 таблиц, 4 представления, 8 функций, 3 триггера и 26 индексов, включая системные индексы первичных и уникальных ключей. Демонстрационный файл queries.sql выполняется без ошибок.

```

docker exec lan_traffic_postgres psql -U postgres -d lan_traffic_db -c "\dt"
docker exec lan_traffic_postgres psql -U postgres -d lan_traffic_db -c "\df"
docker exec lan_traffic_postgres psql -U postgres -d lan_traffic_db -f /tmp/queries.sql

```

13. Соответствие требованиям задания

Требование	Выполнение
Не менее 5 логически связанных таблиц	Реализовано 7 таблиц.
Первичные и внешние ключи	Реализованы во всех основных таблицах.
Тестовое наполнение	Реализовано в sql/002_seed.sql.
Не менее 10 SQL-запросов	Реализовано 20 запросов в sql/queries.sql.
Ограничения NOT NULL, UNIQUE, CHECK	Реализованы в SQL-скриптах.
Индексы	Реализованы для связей, фильтрации и аналитики.
HAVING, CTE, ORDER BY LIMIT	Присутствуют в демонстрационных запросах.
Docker Compose	Реализован запуск PostgreSQL через docker-compose.yml.
PL/pgSQL функции и триггеры	Реализованы функции и триггеры серверной логики.

14. Заключение

В ходе выполнения расчетно-графической работы была спроектирована и реализована база данных для учета сетевой топологии и передачи трафика в локальной сети. База данных позволяет хранить сведения о хостах, коммутаторах, портах, подключениях, MAC-таблице, Ethernet-кадрах и пути их прохождения.

Реализованная схема содержит ограничения целостности, индексы для ускорения запросов, представления для аналитики, функции на SQL и PL/pgSQL, а также триггеры для контроля бизнес-правил и автоматического обновления MAC-таблицы. Для удобного запуска и проверки используется Docker Compose с автоматической инициализацией PostgreSQL.

Разработанная база данных соответствует требованиям задания и может быть использована как учебная модель системы мониторинга и учета событий в локальной сети.